calls to another library. You can specify which HTTP library is used; here I'm using webmock, but that's an implementation detail and this is the last time I'm going to talk about webmock.

The third option, c.configure_rspec_metadata!, configures the RSpec metadata. This allows you to specify VCR as a metadata option in the spec, as you just saw. When you declare the test, if you declare :vcr as metadata, the cassette is automatically named using the names of the describe block as the directory name and the name of the it block as a file name. If you want to pass options to the VCR call, change the metadata declaration from :vcr to vcr: {options}.

The fourth, and last, option causes VCR to ignore requests back to localhost, which can be a problem specifically because Capybara JavaScript drivers will ping the server on localhost, and you want to just let that through.

Within a spec with VCR declared, VCR will behave in one of two ways.

It may think that the cassette needs to be recorded. Most commonly this occurs because the cassette doesn't exist yet, but there are ways to tell VCR to overwrite an existing cassette. If VCR is trying to record a cassette, it'll allow all HTTP requests to happen but save all the requests and responses in a single file—which it'll store in the directory specified in the config, and with the file name specified by the spec name or by an option passed to the metadata.

If VCR doesn't believe the cassette needs to be recorded, it'll act in play mode. In play mode any HTTP request that matches a request in the cassette file will be intercepted, and its response will be crafted by the data on the cassette. Any HTTP request that doesn't match a request in the cassette file will trigger an error and a test failure. You can also optionally make VCR fail if a request in the cassette file isn't matched during the test; in this case VCR is behaving like a mock expectation. You can specify this behavior by including the option allow_unused_http_interactions: false to the spec metadata. If multiple requests in the cassette file match, VCR will use them one at a time in sequence as long as there are further requests to the same URL during the test.

Alternatively, you can use the method call VCR.use_cassette inside RSpec specs just as you would in Minitest, which I'll explain next.

VCR and Minitest

To make VCR work in your Minitest environment, you need to add some configuration to the test_helper.rb file: